

Graphs For Image Segmentation and Image Processing

By:

Joseph Craft

Declan Sheehan

Dr. Anderson

COSC 320-001

11 December, 2019

Abstract

Image segmentation and processing is key to having a computer formulate an “understanding” of the contents of an image. This has a plethora of applications in different lines of work from traffic data gathering to fingerprint recognition. By creating a relatively-basic image segmentation program, we can demonstrate how a computer “decides” to discriminate between different areas of the image. Machine learning through images solely relies on advanced methods to *identify* an image and create a schema out of a massive amount of data. Our paper will not go in-depth into machine-learning, but instead it will describe a basic method of image segmentation using C++. The steps in creating a program that segments an image were to go researching into how we could possibly discriminate between two areas of an image, then formulate a method to automate this, and spit out a new image with differing sections shown. We ended up converting an image into an undirected graph, running our segmentation function, and displaying the new image created using opencv. Our results were relatively successful; however, it has issues when an image section closely relates to another (as expected). What we obtained from this was: using graph-theory and powerful coding languages allows for a *useful* program to segment and process images.

Introduction

Processing digital images dates back to the early 1960’s, where it was used to scan over a series of lights that correspond to different levels of charge a basic capacitor will hold.¹ Since then, computer scientists and engineers have developed more efficient, faster, and robust hardware and techniques. But with the somewhat recent development of mass media has brought upon newer and bigger challenges relating to the gathering of mass-amounts of data. Videos and images are now used to gather day to day data from all kinds of sources. One new invention that relies *greatly* on the acquisition and processing large amounts of data is: the self-driving car. This technology uses high-level programming languages to perform all of the computations needed to observe, process, identify and navigate. In this research project, we aim to create & demonstrate a program made to “discern” different sections of an image also using a high-level programming language. Today, image processing is a well known problem with many methods of taking in a 2D/3D image of pixels/voxels and comparing the individual elements in the graph and placing them into a graph of adjacent elements. For our project there are many great resources available for methods of image processing but we are proposing a simplified approach to image segmentation by grouping image elements based on color and proximity. Our paper and project will focus on image segmentation using graph theory. Our method of image segmentation

will use algorithms that calculate the distance between elements and computes the difference in color of pixels to separate the graph into multiple disjoint sets grouped into a forest of similar elements. The used algorithm will determine if the pixels should be grouped into the same disjoint set based on color and distance.

For our project we needed to apply a gaussian filter to the image to eliminate any data artifacts inside of the image and assist in the calculation of weights for edges. Our method uses a standard library from opencv, but follows the logic in Figure 1 to perform the operation.

The Gaussian blur is a type of image-blurring filter that uses a Gaussian function (which also expresses the normal distribution in statistics) for calculating the transformation to apply to each pixel in the image. The formula of a Gaussian function in one dimension is

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

In two dimensions, it is the product of two such Gaussian functions, one in each dimension:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad [1][2][3]$$

Figure 1: Mathematical representation of Gaussian Blur

```
#include <opencv2/imgproc.hpp>
```

Returns Gaussian filter coefficients.

The function computes and returns the `ksize` × 1 matrix of Gaussian filter coefficients:

$$G_i = \alpha * e^{-\frac{(i-(ksize-1)/2)^2}{2*\sigma^2}},$$

where $i = 0..ksize - 1$ and α is the scale factor chosen so that $\sum_i G_i = 1$.

Two of such generated kernels can be passed to `sepFilter2D`. Those functions automatically recognize smoothing kernels (a symmetrical kernel with sum of weights equal to 1) and handle them accordingly. You may also use the higher-level `GaussianBlur`.

Parameters

ksize Aperture size. It should be odd (`ksize mod 2 = 1`) and positive.

sigma Gaussian standard deviation. If it is non-positive, it is computed from `ksize` as `sigma = 0.3*((ksize-1)*0.5 - 1) + 0.8`.

dtype Type of filter coefficients. It can be `CV_32F` or `CV_64F`.

Figure 2: Gaussian Blur Kernel from opencv

Two dimensions [\[edit \]](#)

In the Euclidean plane, if $\mathbf{p} = (p_1, p_2)$ and $\mathbf{q} = (q_1, q_2)$ then the distance is given by

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}.$$

This is equivalent to the Pythagorean theorem.

Alternatively, it follows from (2) that if the polar coordinates of the point $\mathbf{p}$ are (r_1, θ_1) and those of $\mathbf{q}$ are (r_2, θ_2) , then the distance between the points is

$$\sqrt{r_1^2 + r_2^2 - 2r_1 r_2 \cos(\theta_1 - \theta_2)}.$$

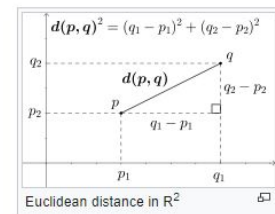


Figure 3: Mathematical representation of Euclidean Distance in 2D

Topic Details

From the algorithm provided above we can begin to address the problem of graph segmentation. Suppose you have an image that is (height x width) and you want to group the elements inside of the image by using the given algorithm above. First, we need to ingest the image and perform a gaussian filter on the image that will blur all of the pixels inside of the image. The methods used are referenced in Figure 1 and the function used to perform this action is referenced in Figure 2. The reason for blurring the image is to remove artifacts from the image and smooth the image to make it easier to compute the weight of each edge.

Second, we need to create the undirected graph $G = (V, E)$ where the graph consists of a lattice connecting vertices vertically and horizontally to adjacent pixels in the image by an edge. Each vertex V represents each individual pixel inside of the image and the edges E is composed of two vertices (v_i, v_j) connecting them in the graph. The final variable we need to address is the weight w that will be used to represent the distance between two vertices inside of the graph. For our methods to get the weight of a pixel pix which is a vector containing a pixel (r, g, b) values we call the function $Diff(image, q_1, p_1, q_2, p_2)$. The elements (p, q) are used to represent the (x, y) coordinate of the pixel inside the image. Once we get the elements $C_1 = (p_1, q_1)$ and $C_2 = (p_2, q_2)$ we can return the $\sqrt{(C_1 - C_2) \cdot (C_1 - C_2)}$. This function was modeled after the algorithm referenced in Figure 3. This is called the Euclidean Distance method to compute the weight of edges. The graph is constructed making $O(n)$ edges for an image with n pixels.²

```
function NN2(p,q) {
  C := {p};
  nn := p;
  while (C ≠ ∅ and termination condition
    not yet met) {
    p := the point of C minimizing dist(q,p);
    C := C - {p};
    for each undiscovered r in N[p] {
      Mark r discovered;
      C := C + {r};
      if (dist(q, r) < dist(q, nn)) nn := r;
    }
  }
  return(nn);
}
```

Figure 4: Approximate Nearest Neighbor (ANN) Algorithm

Third, for the graph we are not just creating graph segments where each vertex is connected to its neighbors, but instead are calling a function which uses the ANN algorithm in

Figure 4 to connect a vertex to its nearest neighbor. This provides an advantage over other methods as it will be able to iterate over neighboring pixels to create segments and approximate which neighbors are within range to be considered similar according to weight. Each segment is broken up into a disjoint set called a forest and each of the elements inside of the disjoint sets will be similar to other elements based on a threshold that is given by the user. Once the forest is constructed, every pixel inside of a forest is re-colored based on the average color of all pixels inside of that forest. This makes it easy to see each individual segment inside of the image. The run time of the segmentation methods takes a total of $O(n \log(n))$ time.²

Conclusion

Image segmentation has come a long way since its first use in linear cameras.¹ Currently, it is used to gather exponentially more data, and has applications and an innumerable amount of fields ranging from road-use, cyber-security, artificial intelligence, and medicine. When studying and implementing a basic image segmentation program, we discovered that there are many steps and methods of segmenting an image. For example, basic segmentation functions that we could have used are: Prim's algorithm, or Kruskal's algorithm, or BFS. Instead, we chose to compute the Euclidean Distance plus color difference between different pixels which gives us a weight. We then use the weight to discriminate between the different regions of the image and use an Approximate Nearest Neighbor algorithm to perform segmentation in the image using the given user threshold and edge weights. The program runs well-enough that we could say it was a success. Of course, it does not properly divide every differing pixel, but for the sake of this demonstration, it functions properly.

This project was educational for both of us, as we learned about multiple methods of processing an image using graph-based tools (along with the mathematics behind it), and became more familiar with C++ libraries. We also got valuable experience using other non-standard C++ libraries with opencv. Overall, we enjoyed the project (especially the outcome), and are now even more familiar with writing research on computer-science based topics; which will be useful for the time to come.

Bibliography

- [1] Williams JB. *The Electronics Revolution: Inventing the Future*. Cham: Springer; 2017.
- [2] Felzenszwalb PF, Huttenlocher DP. *Efficient Graph-Based Image Segmentation*. International Journal of Computer Vision. 2004;59(2):167-181.
doi:10.1023/b:visi.0000022288.19776.77
- [3] Arya, Sunil, and David M. Mount. "Approximate Nearest Neighbor Queries in Fixed Dimensions." SODA. Vol. 93. 1993.

Reference for ‘Figure 1’

- [1] Gaussian blur. Wikipedia. https://en.wikipedia.org/wiki/Gaussian_blur. Published November 25, 2019. Accessed December 10, 2019.
- [2] Shapiro, L. G. & Stockman, G. C: "Computer Vision", page 137, 150. Prentice Hall, 2001
- [3] Mark S. Nixon and Alberto S. Aguado. *Feature Extraction and Image Processing*. Academic Press, 2008, p. 88.
- [4] R.A. Haddad and A.N. Akansu, "A Class of Fast Gaussian Binomial Filters for Speech and Image Processing," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 39, pp 723-727, March 1991.

Reference for ‘Figure 2’

- [1] Smoothing Images. OpenCV.
https://docs.opencv.org/master/d4/d13/tutorial_py_filtering.html. Accessed December 10, 2019.

Reference for ‘Figure 3’

- [1] Euclidean distance. Wikipedia. https://en.wikipedia.org/wiki/Euclidean_distance. Published September 5, 2019. Accessed December 10, 2019.

Reference for ‘Figure 4’

- [1] Arya, Sunil, and David M. Mount. "Approximate Nearest Neighbor Queries in Fixed Dimensions." SODA. Vol. 93. 1993.

Appendix

Segmented: (Threshold, Min_Size)
Note: (0, 0) is the original image.

(0, 0)



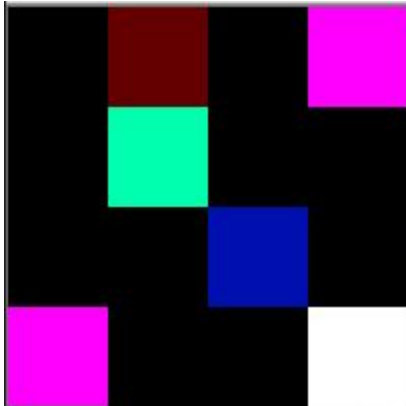
(1000, 20)



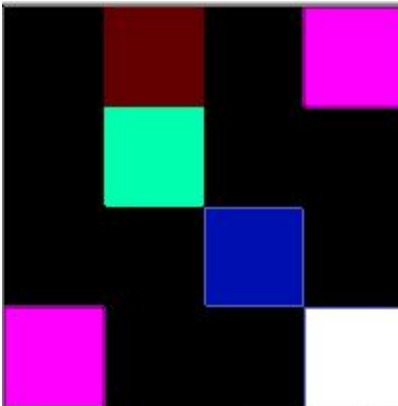
(4000, 100)



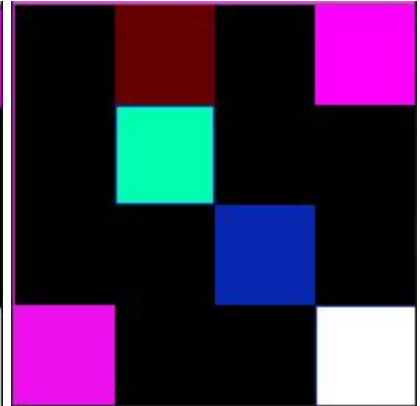
(0, 0)



(1000, 20)



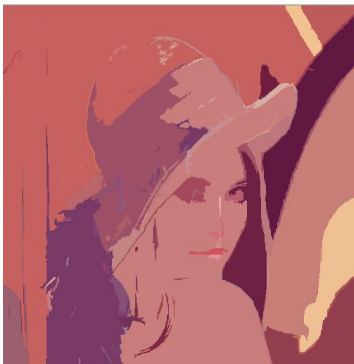
(4000, 1000)



(0, 0)



(1000, 20)



(4000, 100)

